

DATA ANALYSIS AND MACHINE LEARNING REPORT

Instagram Pav-Bhaji Image Classification

Using Text Metadata

Classification of Instagram Posts as Pav-Bhaji or Not Pav-Bhaji

Submitted By: Aaditya Anand

Uid: 23BCS10446

University: Chandigarh University

Program: Computer Science and Engineering

Dataset: Instagram Pav-Bhaji Metadata Dataset

Classification Type: Binary Classification

Primary Model: TF-IDF + Logistic Regression

Machine Learning / Data Preprocessing Challenge

September 2026

Abstract

This report presents a text-based machine learning approach for classifying Instagram posts into two categories: *Pav-Bhaji* and *Not Pav-Bhaji*. The objective was to determine whether the available textual metadata associated with an Instagram image could provide useful signals for classification without directly processing image pixels.

The provided JSON dataset contained 1,500 Instagram posts with multiple metadata fields, including captions, tags, URLs, timestamps, engagement information, and other attributes. After matching the posts with the supplied labeled image folders, 452 posts were identified as usable labeled examples, including 269 Not Pav-Bhaji examples and 183 Pav-Bhaji examples.

For classification, the caption and tag information were combined and preprocessed using lowercase conversion, URL and mention removal, hashtag symbol removal, punctuation normalization, and whitespace normalization. The resulting text was converted into numerical features using TF-IDF with unigram and bigram features. Three machine learning approaches were evaluated: Logistic Regression, Linear Support Vector Machine, and Multinomial Naive Bayes.

For the selected 80:20 stratified split, the Logistic Regression model achieved an accuracy of 64.84% and an F1-score of 64.44%. A 5-fold stratified cross-validation experiment produced a mean accuracy of 68.13% with a standard deviation of 4.44%, and a mean F1-score of 65.98% with a standard deviation of 4.75%.

The results indicate that Instagram textual metadata contains useful information for distinguishing the two classes. However, the relatively small labeled dataset, noisy metadata, collection-related bias, and absence of visual information limit the generalization of the approach to completely independent real-world images.

Keywords: Text Classification, Instagram Metadata, TF-IDF, Logistic Regression, Machine Learning, Pav-Bhaji, Natural Language Processing

Contents

1	Introduction	5
2	Objective	5
3	Problem Formulation	6
4	Dataset Overview	6
5	Labeled Dataset Construction	8
6	Dataset Summary	8
7	Exploratory Data Analysis	10
8	Textual Feature Identification	10
9	Caption Extraction	11
10	Tag Extraction	11
11	Text Quality Analysis	12
12	Text Preprocessing	12
13	Preprocessing Pipeline	14
14	Relevance of Selected Features	15
15	Train-Test Splitting	16
16	TF-IDF Feature Extraction	16
17	TF-IDF Configuration	17
18	Logistic Regression	17

19 Linear Support Vector Machine	18
20 Multinomial Naive Bayes	18
21 Model Training	19
22 Evaluation Metrics	20
22.1 Accuracy	20
22.2 Precision	20
22.3 Recall	20
22.4 F1-Score	20
23 Model Comparison	21
24 Cross-Validation	22
25 Cross-Validation Results	22
26 Final Model Selection	23
27 Confusion Matrix Analysis	24
28 Feature Coefficient Analysis	25
28.1 Features Associated with Pav-Bhaji	25
28.2 Features Associated with Not Pav-Bhaji	26
29 Interpretation of Model Features	27
30 Target-Word Dependence	28
31 Limitations	29
31.1 Limited Number of Labeled Samples	29
31.2 Noisy Social-Media Metadata	29
31.3 Collection-Related Bias	29
31.4 Absence of Visual Features	29

31.5 Limited Generalization	30
32 Data Leakage Considerations	30
33 Final Prediction Pipeline	31
33.1 Prediction Function	31
34 Prediction Examples	32
34.1 Example 1: Pav-Bhaji Related Input	32
34.2 Example 2: Non-Pav-Bhaji Input	32
35 Future Improvements	33
35.1 Larger and More Diverse Dataset	33
35.2 Improved Label Quality	33
35.3 Advanced Text Representations	33
35.4 Visual Feature Integration	34
35.5 Robust Evaluation	34
36 Conclusion	34
37 References	35
A Appendix A: Main Implementation Workflow	35
B Appendix B: Final Model Configuration	37
C Appendix C: Performance Summary	37

1. Introduction

Social media platforms such as Instagram contain a large amount of user-generated multimedia content. Each post can contain both visual information and associated textual metadata such as captions and hashtags. While image-based classification generally relies on computer vision techniques, textual metadata can also provide useful contextual information about the content of an image.

This project focuses on the classification of Instagram posts into two categories: *Pav-Bhaji* and *Not Pav-Bhaji*. The key constraint of the task is that the classification must be performed using textual metadata rather than directly analyzing image pixels.

The problem is therefore formulated as a binary text classification problem. The available caption and tag information is processed and transformed into numerical features using Term Frequency–Inverse Document Frequency (TF-IDF). Several traditional machine learning algorithms are then evaluated to determine their ability to distinguish between the two classes.

The overall workflow consists of dataset exploration, label matching, text preprocessing, feature extraction, model training, model comparison, cross-validation, and final model analysis.

2. Objective

The primary objective of this project is to develop a machine learning pipeline that predicts whether an Instagram post belongs to the *Pav-Bhaji* class or the *Not Pav-Bhaji* class using only available textual metadata.

The specific objectives are:

1. Load and inspect the provided Instagram metadata stored in JSON format.
2. Identify the textual fields relevant to classification, primarily captions and tags.
3. Match the Instagram metadata with the supplied labeled image folders to construct a supervised learning dataset.
4. Clean and normalize the textual information while preserving meaningful content.
5. Convert the processed text into numerical representations using TF-IDF.
6. Train and compare multiple machine learning classification models.

7. Evaluate the models using accuracy, precision, recall, and F1-score.
8. Perform stratified cross-validation to obtain a more stable estimate of model performance.
9. Analyze the confusion matrix and important textual features learned by the final model.
10. Develop a reusable prediction pipeline capable of classifying new caption and tag inputs.

3. Problem Formulation

The task can be represented as a binary classification problem.

Let X represent the textual metadata of an Instagram post and y represent its corresponding class label:

$$y \in \{0, 1\} \quad (1)$$

where:

- $y = 0$ represents *Not Pav-Bhaji*.
- $y = 1$ represents *Pav-Bhaji*.

For each post, the caption and tags are combined to form a single textual representation:

$$X = \text{Caption} + \text{Tags} \quad (2)$$

The machine learning model learns a relationship between the textual representation X and the corresponding class label y .

The final prediction system can therefore be represented as:

$$\text{Caption} + \text{Tags} \rightarrow \text{Text Preprocessing} \rightarrow \text{TF-IDF} \rightarrow \text{Classifier} \rightarrow \text{Class Prediction} \quad (3)$$

4. Dataset Overview

The provided dataset consists of Instagram post metadata stored in a JSON file named `pavbhaji.json`. The dataset contains information about 1,500 Instagram posts.

Each post contains multiple metadata attributes. The available fields include information related to image dimensions, captions, tags, engagement, identification, location, timestamps, URLs, and other Instagram attributes.

The JSON dataset contains the following 18 fields:

- dimensions
- display_url
- edge_liked_by
- edge_media_preview_like
- edge_media_to_caption
- edge_media_to_comment
- id
- is_video
- location
- owner
- shortcode
- tags
- taken_at_timestamp
- thumbnail_resources
- thumbnail_src
- urls
- video_view_count
- comments_disabled

Although the JSON file contains 1,500 posts, only the posts that could be matched with the supplied labeled image folders were used for supervised classification.

5. Labeled Dataset Construction

The supplied image dataset was organized into two folders representing the two target classes:

- Folder 0: Not Pav-Bhaji
- Folder 1: Pav-Bhaji

The image identifiers were matched against the available Instagram metadata, using information such as the image URL and Instagram shortcode.

This process resulted in 452 successfully matched and labeled posts.

The resulting class distribution is shown in Table 1.

Table 1: Class distribution of the labeled dataset

Class	Label	Number of Samples
Not Pav-Bhaji	0	269
Pav-Bhaji	1	183
Total		452

The dataset contains more Not Pav-Bhaji examples than Pav-Bhaji examples. Therefore, the classes are not perfectly balanced. To account for this difference during model training, class weighting was used for the Logistic Regression and Linear SVM models.

6. Dataset Summary

Table 2 provides a summary of the main dataset characteristics identified during exploratory analysis.

Table 2: Dataset summary

Property	Value
Total Instagram posts in JSON	1,500
Total metadata fields	18
Successfully labeled posts	452
Not Pav-Bhaji samples	269
Pav-Bhaji samples	183
Empty captions	1
Posts with empty tags	18
Posts with completely empty combined text	1
Average caption length	420.92 characters
Average combined text length	625.93 characters

The analysis shows that the combination of captions and tags provides a larger textual representation than captions alone. The average length of the combined textual representation was approximately 625.93 characters, compared with approximately 420.92 characters for captions alone.

7. Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed before model development to understand the structure and quality of the available metadata.

The initial JSON dataset contained 1,500 Instagram posts. Each post included multiple attributes, but not every post had a corresponding image label. Therefore, label matching was performed before constructing the final supervised learning dataset.

The analysis identified 452 posts with successfully matched labels. These 452 observations were subsequently used for model development and evaluation.

The class distribution was:

- 269 samples belonged to the Not Pav-Bhaji class.
- 183 samples belonged to the Pav-Bhaji class.

This corresponds to approximately 59.51% Not Pav-Bhaji and 40.49% Pav-Bhaji samples.

The class distribution is therefore moderately imbalanced. Rather than discarding samples from the majority class, the available labeled data was retained. Class weighting was subsequently used in the Logistic Regression and Linear SVM models.

8. Textual Feature Identification

The JSON metadata contains several fields, but the classification task was restricted to textual metadata relevant to the content of each Instagram post.

Two fields were selected as the primary input features:

1. **Caption**
2. **Tags / Hashtags**

The caption contains natural-language descriptions written by the user, while tags provide additional keywords associated with the post.

The two sources were combined into a single textual representation:

$$\text{Combined Text} = \text{Caption Text} + \text{Tag Text} \quad (4)$$

This representation was selected because tags can provide additional information when the caption itself is short or ambiguous.

Other metadata fields such as likes, comments, timestamps, URLs, image dimensions, and video information were not used as direct textual classification features.

9. Caption Extraction

The caption information in the JSON file was stored in a nested structure. The relevant text was therefore extracted from the nested `edge_media_to_caption` field.

The extraction procedure can be represented conceptually as:

```
edge_media_to_caption
|
+-- edges
|
+-- node
|
+-- text
```

The first available caption text was extracted from this structure. If a caption was unavailable, an empty string was returned.

The extracted caption was stored in a new dataframe column named `caption_text`.

10. Tag Extraction

The `tags` field contained a list of tags associated with each Instagram post.

The list of tags was converted into a single space-separated string. For example:

```
['food', 'mumbai', 'pavbhaji', 'streetfood']
```

was transformed into:

```
food mumbai pavbhaji streetfood
```

The resulting text was stored in a new dataframe column named `tags_text`.

The caption and tag strings were then concatenated and converted to lowercase to produce the final `combined_text` representation.

11. Text Quality Analysis

A basic quality check was performed on the extracted textual information.

The analysis found:

- 1 post with an empty caption.
- 18 posts with empty tag information.
- 1 post with completely empty combined text.

The average caption length was approximately 420.92 characters, whereas the average combined caption-and-tag length was approximately 625.93 characters.

This indicates that adding tags increases the amount of available textual information for classification.

Missing textual values were not replaced with arbitrary content. Instead, empty values were represented as empty strings so that the preprocessing and TF-IDF stages could handle them consistently.

12. Text Preprocessing

Raw social-media text commonly contains URLs, user mentions, hashtags, punctuation, inconsistent capitalization, and unnecessary whitespace. Preprocessing was therefore applied before feature extraction.

The following operations were performed:

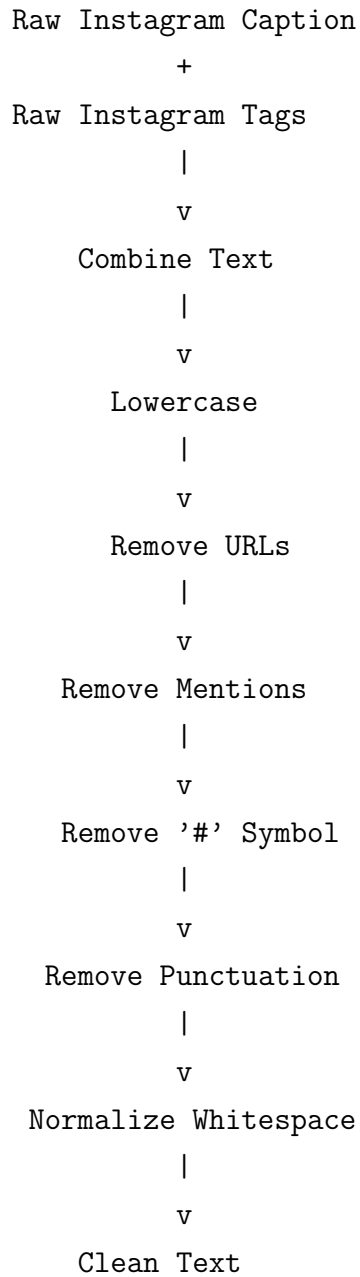
1. Convert all text to lowercase.
2. Remove URLs beginning with patterns such as `http` or `www`.
3. Remove user mentions represented using the `@` symbol.
4. Remove the hashtag symbol while retaining the word following it.
5. Remove punctuation and unnecessary special characters.
6. Normalize repeated whitespace.
7. Preserve Unicode characters so that non-English textual content is not unnecessarily removed.

The preprocessing function used in the implementation was:

```
1 def clean_text(text):
2     text = str(text).lower()
3
4     text = re.sub(
5         r"http\S+|www\S+",
6         " ",
7         text
8     )
9
10    text = re.sub(
11        r"@w+",
12        " ",
13        text
14    )
15
16    text = re.sub(
17        r"#",
18        "",
19        text
20    )
21
22    text = re.sub(
23        r"[\w\s]",
24        " ",
25        text,
26        flags=re.UNICODE
27    )
28
29    text = re.sub(
30        r"\s+",
31        " ",
32        text
33    ).strip()
34
35    return text
```

13. Preprocessing Pipeline

The complete preprocessing process can be summarized as follows:



The resulting clean text was stored in the dataframe column `clean_text`.

This cleaned representation was then passed to the TF-IDF feature extraction stage.

14. Relevance of Selected Features

The decision to use caption and tags was further examined by comparing a caption-only representation with the combined caption-and-tags representation.

The caption-only experiment used the same train-test split and Logistic Regression methodology as the combined representation.

The results were:

Table 3: Comparison of caption-only and caption-plus-tags representations

Representation	Accuracy	Precision	Recall	F1
Caption Only	63.74%	54.35%	67.57%	60.24%
Caption + Tags	64.84%	54.72%	78.38%	64.44%

The combined caption-and-tags representation produced higher accuracy, precision, recall, and F1-score on the selected test split.

The increase in recall is particularly notable. The combined representation identified a larger proportion of the actual Pav-Bhaji examples in the test set.

These results support the use of both captions and tags as the primary text representation for the final model.

15. Train-Test Splitting

After preprocessing, the labeled dataset was divided into training and testing subsets.

An 80:20 train-test split was used. Stratification was applied to preserve the relative proportion of the two target classes in both subsets.

The resulting distribution is shown in Table 4.

Table 4: Training and testing dataset distribution

Class	Training	Testing	Total
Not Pav-Bhaji	215	54	269
Pav-Bhaji	146	37	183
Total	361	91	452

Stratification was used because the dataset contains a moderate difference between the number of Not Pav-Bhaji and Pav-Bhaji examples. This ensures that both classes are represented in similar proportions in the training and testing subsets.

16. TF-IDF Feature Extraction

Machine learning algorithms cannot directly process raw textual data. Therefore, the cleaned Instagram text was transformed into numerical feature vectors using Term Frequency–Inverse Document Frequency (TF-IDF).

TF-IDF assigns a numerical weight to a term based on its frequency within a document and its frequency across the complete collection of documents.

The term frequency component can be expressed as:

$$TF(t, d) = \frac{\text{Number of occurrences of term } t \text{ in document } d}{\text{Total number of terms in document } d} \quad (5)$$

The inverse document frequency component can be expressed as:

$$IDF(t) = \log \left(\frac{N}{1 + DF(t)} \right) \quad (6)$$

where:

- N is the total number of documents.

- $DF(t)$ is the number of documents containing term t .

The TF-IDF representation is then calculated as:

$$TFIDF(t, d) = TF(t, d) \times IDF(t) \quad (7)$$

In this project, both individual words and two-word combinations were considered. This allows the model to learn not only individual terms such as `pavbhaji`, but also contextual combinations such as `homemade pavbhaji`.

17. TF-IDF Configuration

The TF-IDF vectorizer was configured using the following parameters:

Table 5: TF-IDF configuration

Parameter	Value
Stop words	English
N-gram range	(1, 2)
Maximum features	2,000
Minimum document frequency	2

The TF-IDF vectorizer was fitted only on the training text and subsequently used to transform the testing text.

This separation is important because fitting the vectorizer on the complete dataset before splitting could allow information from the test set to influence the feature extraction process.

18. Logistic Regression

Logistic Regression was selected as one of the primary models because it is a widely used algorithm for binary classification and works effectively with high-dimensional sparse text representations such as TF-IDF.

For binary classification, Logistic Regression estimates the probability of a sample belonging to the positive class using the sigmoid function:

$$P(y = 1|x) = \frac{1}{1 + e^{-(w^T x + b)}} \quad (8)$$

where:

- x represents the TF-IDF feature vector.
- w represents the learned feature weights.
- b represents the model intercept.

A class-weighted configuration was used to reduce the influence of the moderate class imbalance.

The implementation used:

```
1 LogisticRegression(  
2     class_weight="balanced",  
3     random_state=42,  
4     max_iter=1000  
5 )
```

19. Linear Support Vector Machine

A Linear Support Vector Machine (Linear SVM) was evaluated as a second classification approach.

Linear SVM attempts to find a decision boundary that separates the two classes while maximizing the margin between them.

The Linear SVM implementation used:

```
1 LinearSVC(  
2     class_weight="balanced",  
3     random_state=42  
4 )
```

Class weighting was again used to account for the difference in class frequencies.

20. Multinomial Naive Bayes

Multinomial Naive Bayes was selected as a third baseline model because it is commonly used for text classification problems.

The algorithm estimates the probability of each class based on the occurrence of the observed textual features. It assumes conditional independence between features given the class.

The implementation used:

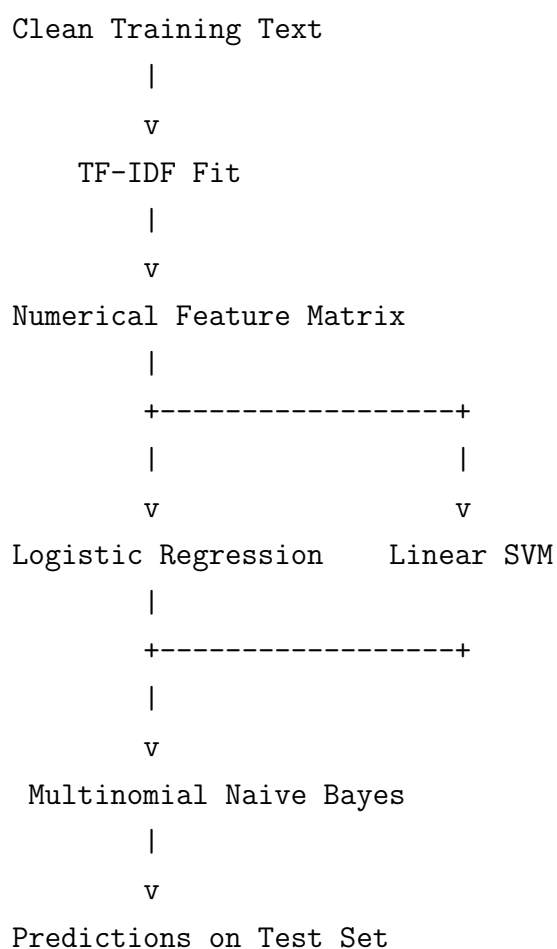
```
1 MultinomialNB()
```

This model provides a useful baseline against which the Logistic Regression and Linear SVM approaches can be compared.

21. Model Training

All three models were trained using the TF-IDF representation generated from the training dataset.

The general training process was:



The TF-IDF feature matrix generated for the training data contained 361 observations

and up to 2,000 textual features. The same fitted vectorizer was used to transform the 91 test observations.

22. Evaluation Metrics

The models were evaluated using four primary metrics: accuracy, precision, recall, and F1-score.

22.1 Accuracy

Accuracy measures the proportion of correctly classified observations:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (9)$$

22.2 Precision

Precision measures the proportion of predicted positive examples that are actually positive:

$$Precision = \frac{TP}{TP + FP} \quad (10)$$

22.3 Recall

Recall measures the proportion of actual positive examples that were correctly identified:

$$Recall = \frac{TP}{TP + FN} \quad (11)$$

22.4 F1-Score

F1-score is the harmonic mean of precision and recall:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (12)$$

These metrics provide a more complete evaluation than accuracy alone, particularly because the dataset contains two classes with different sample counts.

23. Model Comparison

The three evaluated models were compared using the same 80:20 stratified test split.

The resulting performance is shown in Table 6.

Table 6: Performance comparison of evaluated models

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	64.84%	54.72%	78.38%	64.44%
Linear SVM	60.44%	51.02%	67.57%	58.14%
Multinomial NB	63.74%	53.70%	78.38%	63.74%

For the selected test split, Logistic Regression achieved the highest accuracy and F1-score among the three evaluated models. Its recall for the Pav-Bhaji class was 78.38%, indicating that it correctly identified a large proportion of the actual Pav-Bhaji examples in the test set.

The results also show that Multinomial Naive Bayes produced a similar recall to Logistic Regression, while Linear SVM produced lower values across the reported metrics for this split.

The test-set comparison should be interpreted together with the subsequent cross-validation results because the labeled dataset contains only 452 observations.

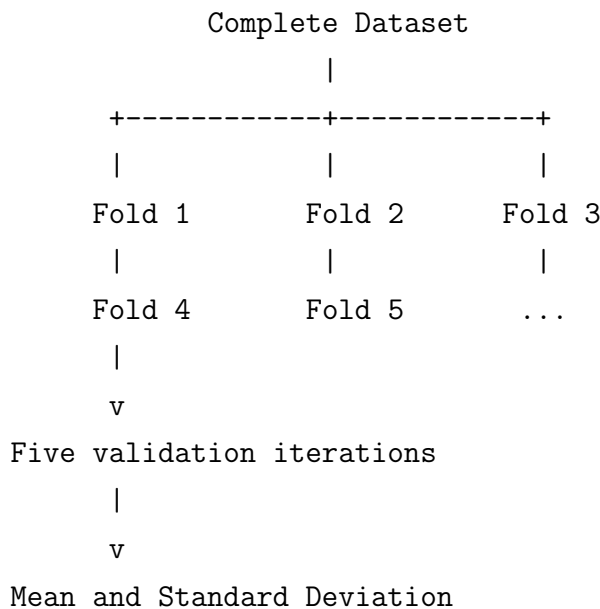
24. Cross-Validation

A single train-test split can produce performance estimates that depend on which observations happen to be included in the test set. This is especially important for the present dataset because only 452 labeled observations were available.

To obtain a more stable estimate of model performance, 5-fold stratified cross-validation was performed for the Logistic Regression pipeline.

In 5-fold cross-validation, the labeled dataset is divided into five subsets. The model is trained using four subsets and evaluated on the remaining subset. This process is repeated until every subset has been used as the validation set.

The process can be represented as:



A stratified version was used so that the relative class distribution was preserved across the folds.

25. Cross-Validation Results

The 5-fold stratified cross-validation results for the final Logistic Regression pipeline are presented in Table 7.

Table 7: 5-fold stratified cross-validation results

Metric	Mean	Standard Deviation
Accuracy	68.13%	4.44%
Precision	58.15%	4.33%
Recall	76.46%	6.50%
F1-Score	65.98%	4.75%

The model obtained a mean accuracy of 68.13% across the five folds, with a standard deviation of 4.44 percentage points.

The mean F1-score was 65.98%, with a standard deviation of 4.75 percentage points.

The relatively moderate standard deviations indicate that model performance varied across the different folds, which is expected given the relatively small size and variability of the labeled dataset.

For this reason, the cross-validation results provide a more stable description of expected performance than relying solely on one particular train-test split.

26. Final Model Selection

Based on the experiments conducted, the final classification pipeline uses TF-IDF feature extraction followed by Logistic Regression.

The final pipeline is:

Caption + Tags $\rightarrow$ Text Cleaning $\rightarrow$ TF-IDF $\rightarrow$ Logistic Regression $\rightarrow$ Prediction (13)

The Logistic Regression model was selected because, on the evaluated 80:20 test split, it produced the highest accuracy and F1-score among the three tested algorithms.

The final test-set results were:

- Accuracy: **64.84%**
- Precision for Pav-Bhaji: **54.72%**
- Recall for Pav-Bhaji: **78.38%**
- F1-score for Pav-Bhaji: **64.44%**

The cross-validation results are also considered when interpreting the performance because the test split was used during iterative experimentation. The 5-fold cross-validation accuracy of 68.13% provides a more stable estimate of performance across different subsets of the labeled data.

27. Confusion Matrix Analysis

The final Logistic Regression model was evaluated on the 91 observations in the test set.

The resulting confusion matrix was:

Table 8: Confusion matrix of the final model

	Predicted Not Pav-Bhaji	Predicted Pav-Bhaji
Actual Not Pav-Bhaji	30	24
Actual Pav-Bhaji	8	29

The four entries of the confusion matrix can be interpreted as follows:

- **True Negatives (TN) = 30:** 30 Not Pav-Bhaji posts were correctly classified as Not Pav-Bhaji.
- **False Positives (FP) = 24:** 24 Not Pav-Bhaji posts were incorrectly classified as Pav-Bhaji.
- **False Negatives (FN) = 8:** 8 Pav-Bhaji posts were incorrectly classified as Not Pav-Bhaji.
- **True Positives (TP) = 29:** 29 Pav-Bhaji posts were correctly classified as Pav-Bhaji.

The model therefore correctly classified:

$$30 + 29 = 59 \quad (14)$$

out of 91 test observations.

The resulting accuracy is:

$$Accuracy = \frac{59}{91} \times 100 = 64.84\% \quad (15)$$

The model identified 29 of the 37 actual Pav-Bhaji examples in the test set. This corresponds to a Pav-Bhaji recall of:

$$Recall = \frac{29}{29 + 8} \times 100 = 78.38\% \quad (16)$$

28. Feature Coefficient Analysis

Logistic Regression provides a coefficient for each TF-IDF feature. These coefficients can be inspected to understand which terms were more strongly associated with each class in the training data.

A positive coefficient indicates an association with the positive class, which in this project is the Pav-Bhaji class. A negative coefficient indicates an association with the Not Pav-Bhaji class.

It is important to note that these coefficients represent statistical associations learned from the available dataset. They should not be interpreted as universal rules about Pav-Bhaji.

28.1 Features Associated with Pav-Bhaji

The strongest positive features learned by the Logistic Regression model included the terms shown in Table 9.

Table 9: Features with strongest positive Logistic Regression coefficients

Feature	Coefficient
pavbhaji	1.3929
foodlover	1.2630
pavbhaji pavbhaji	0.8966
foodgasm	0.8576
pav	0.8105
monsoon	0.7796
homemade	0.7708
vegetables	0.7222
bhukkad	0.6945
follow	0.6748
breakfast	0.6702
mumbaifood	0.6349
butter	0.6270
special	0.6141
likeforlikes	0.6140
bhaji	0.6107
foodphotography	0.6090
street	0.5996
buttery	0.5991
homecooking	0.5869

Several of these features are directly or indirectly related to food, homemade preparation, vegetables, butter, street food, and Mumbai food culture.

The term `pavbhaji` received the largest positive coefficient in the model. This indicates that its occurrence was strongly associated with the Pav-Bhaji class in the training data.

28.2 Features Associated with Not Pav-Bhaji

The strongest negative features learned by the model included:

Table 10: Features with strongest negative Logistic Regression coefficients

Feature	Coefficient
fondue	-0.8042
vadapav	-0.7586
bhelpuri	-0.7266
panipuri	-0.7062
homemade pavbhaji	-0.6489
pizza	-0.5609
mumbaidiaries	-0.4982
foodforlife	-0.4919
dosa	-0.4743
delhigram	-0.4609
masaladosa	-0.4496
foodislife	-0.4487
gobimanchurian	-0.4445
snack	-0.4441
kolkatadiaries	-0.4363
golgappa	-0.4204
foodofmumbai	-0.4074
chicken	-0.4070
delhibloggers	-0.3995
weekendvibes	-0.3961

The presence of terms such as **pizza**, **dosa**, **chicken**, **bhelpuri**, and **panipuri** indicates that the model learned associations with other food categories that occurred in the labeled examples.

Some learned associations may appear counterintuitive. This is expected in a relatively small and noisy social-media dataset, where hashtags and captions may mention multiple foods or may not precisely describe the visual content of the image.

29. Interpretation of Model Features

The feature coefficient analysis provides an interpretable view of the classification model.

Some learned associations are intuitive. For example, terms such as **pavbhaji**, **bhaji**, **butter**, **vegetables**, and **homecooking** provide textual signals that may be relevant to Pav-Bhaji-related posts.

However, the model also learned associations that are specific to the available dataset. For example, terms associated with other foods such as `pizza`, `dosa`, and `chicken` appear among the features associated with the Not Pav-Bhaji class.

This demonstrates that the model is learning statistical patterns from the training data rather than understanding the image itself.

Consequently, feature coefficients should be interpreted as evidence of dataset-level associations rather than as causal relationships.

30. Target-Word Dependence

The feature analysis showed that `pavbhaji` was the strongest positive feature in the Logistic Regression model.

This observation is important because the dataset was collected around Pav-Bhaji-related Instagram content. Therefore, the presence of the term `pavbhaji` in captions or tags may provide a strong collection-related signal.

The model is consequently not equivalent to a purely visual classifier. Instead, it learns associations between textual metadata and the supplied labels.

This is an important limitation when considering how the model would perform on completely independent Instagram posts where users may not mention Pav-Bhaji in the caption or tags.

31. Limitations

Although the developed pipeline demonstrates that textual metadata can provide useful signals for Pav-Bhaji classification, several limitations should be considered when interpreting the results.

31.1 Limited Number of Labeled Samples

The original JSON dataset contained 1,500 Instagram posts. However, only 452 posts could be successfully matched with the supplied labeled image folders.

Consequently, the supervised learning experiments were performed on a relatively small dataset. A larger labeled dataset could provide more representative examples and potentially improve generalization.

31.2 Noisy Social-Media Metadata

Instagram captions and tags are user-generated and may contain multiple food names, promotional text, generic food-related terms, or information that does not directly describe the visual content.

For example, a post may mention several different food items in its caption or tags even though only one food item appears in the image.

Therefore, the textual metadata should not always be treated as a perfect description of the image.

31.3 Collection-Related Bias

The term `pavbhaji` was one of the strongest features learned by the Logistic Regression model.

Since the dataset was collected around Pav-Bhaji-related Instagram content, this feature may contain a collection-related signal. As a result, the model may perform differently on independent data where users do not mention Pav-Bhaji in their captions or tags.

31.4 Absence of Visual Features

The primary limitation of this approach is that the actual image pixels were not used.

The system therefore cannot directly determine whether Pav-Bhaji is visually present in an image. Instead, it predicts the class from the associated textual metadata.

For example, an image containing Pav-Bhaji but having an unrelated caption and tags may be difficult for the model to classify correctly.

31.5 Limited Generalization

The reported performance values are specific to the supplied dataset and experimental setup.

They should not be interpreted as a universal measure of the accuracy of text-based Pav-Bhaji classification on arbitrary Instagram data.

32. Data Leakage Considerations

Data leakage was considered during the feature extraction and model training process.

The TF-IDF vectorizer was fitted only on the training portion of the data:

```
1 X_train = vectorizer.fit_transform(X_train_text)
2 X_test = vectorizer.transform(X_test_text)
```

The test data was therefore transformed using the vocabulary and statistics learned from the training data rather than fitting the vectorizer on the complete dataset.

For cross-validation, the TF-IDF transformation and Logistic Regression classifier were placed inside a scikit-learn Pipeline. This ensures that feature extraction is performed separately within each cross-validation training fold.

The cross-validation pipeline was structured as:

```
1 pipeline = Pipeline([
2     ("tfidf", TfidfVectorizer(
3         stop_words="english",
4         ngram_range=(1, 2),
5         max_features=2000,
6         min_df=2
7     )),
8     ("classifier", LogisticRegression(
9         class_weight="balanced",
10        random_state=42,
11        max_iter=1000
12    ))
13 ])
```

13 | 1)

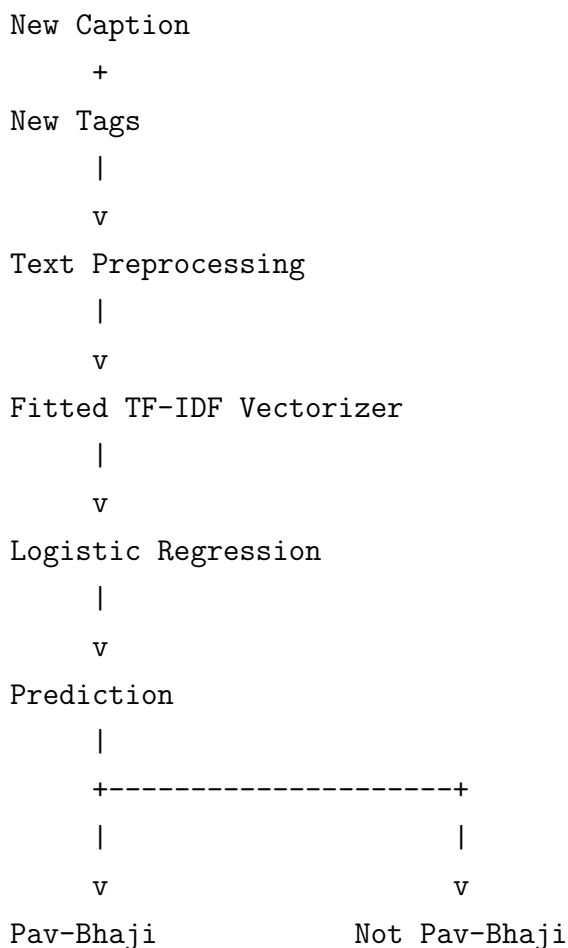
This procedure helps prevent information from the validation portion from being used to construct the TF-IDF representation during each fold.

33. Final Prediction Pipeline

After model evaluation, a reusable prediction function was created to classify new Instagram metadata.

The prediction function accepts a caption and an optional set of tags, combines them, applies the same preprocessing procedure used during training, and passes the resulting text through the trained pipeline.

The final pipeline can be represented as:



33.1 Prediction Function

The implemented prediction function was:

```

1 def predict_pav_bhaji(caption, tags=""):
2
3     text = str(caption) + " " + str(tags)
4
5     text = clean_text(text)
6
7     prediction = final_pipeline.predict([text])[0]
8
9     probability = final_pipeline.predict_proba([text])[0]
10
11     if prediction == 1:
12         label = "Pav-Bhaji"
13     else:
14         label = "Not Pav-Bhaji"
15
16     return label, probability[prediction]

```

34. Prediction Examples

The final prediction function was tested with two example inputs.

34.1 Example 1: Pav-Bhaji Related Input

The first test contained a caption describing homemade Pav-Bhaji with butter and vegetables.

The model produced:

Table 11: Prediction example for Pav-Bhaji-related metadata

Property	Result
Input type	Pav-Bhaji-related food description
Prediction	Pav-Bhaji
Predicted probability	70.99%

34.2 Example 2: Non-Pav-Bhaji Input

The second test contained metadata describing chicken tikka.

The model produced:

Table 12: Prediction example for non-Pav-Bhaji metadata

Property	Result
Input type	Chicken tikka-related food description
Prediction	Not Pav-Bhaji
Predicted probability	63.92%

These examples demonstrate that the trained pipeline can be reused for classification of new textual metadata.

35. Future Improvements

Several improvements could be explored in a future version of the project.

35.1 Larger and More Diverse Dataset

Increasing the number of labeled examples would provide the models with more varied examples of both classes and could improve their ability to generalize.

35.2 Improved Label Quality

Manual verification of labels could reduce noise in the training dataset. This would be particularly useful for social-media posts containing multiple food-related terms.

35.3 Advanced Text Representations

Future experiments could evaluate additional natural language processing representations, including:

- Word embeddings
- Sentence embeddings
- Transformer-based text representations
- Character-level TF-IDF features

These approaches may capture semantic and morphological information that is not fully represented by standard word-level TF-IDF.

35.4 Visual Feature Integration

A more complete image classification system could combine textual metadata with visual features extracted from the image.

For example, a future multimodal system could combine:

$$\text{Visual Features} + \text{Textual Features} \rightarrow \text{Classification} \quad (17)$$

Such an approach would allow the model to use both what is visually present in an image and what the user wrote about the image.

35.5 Robust Evaluation

Future work could use a larger independent test set that is kept completely separate from model development.

Repeated cross-validation and evaluation on an external dataset could also provide stronger evidence about generalization performance.

36. Conclusion

This project developed a complete machine learning pipeline for classifying Instagram posts as *Pav-Bhaji* or *Not Pav-Bhaji* using textual metadata.

The original JSON dataset contained 1,500 Instagram posts, from which 452 posts were successfully matched with the supplied labels. Caption and tag information were selected as the primary input features.

The textual data was cleaned and transformed using TF-IDF with unigram and bigram features. Three machine learning algorithms were evaluated: Logistic Regression, Linear SVM, and Multinomial Naive Bayes.

On the selected 80:20 stratified test split, Logistic Regression achieved an accuracy of 64.84% and an F1-score of 64.44%. The model achieved a recall of 78.38% for the Pav-Bhaji class.

A 5-fold stratified cross-validation experiment provided a broader estimate of performance,

producing a mean accuracy of $68.13\% \pm 4.44\%$ and a mean F1-score of $65.98\% \pm 4.75\%$.

The experiments also demonstrated that combining captions and tags provided more useful information than using captions alone on the evaluated split.

Overall, the results demonstrate that textual metadata contains measurable signals that can be used for Pav-Bhaji classification. At the same time, the small labeled dataset, noisy social-media text, collection-related bias, and absence of visual features represent important limitations.

The developed pipeline provides a reproducible baseline for the given classification task and can be extended in future work using larger datasets, improved labels, advanced text representations, and multimodal visual-textual models.

37. References

1. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
2. Salton, G. and Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513–523.
3. Manning, C. D., Raghavan, P., and Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
4. The supplied Instagram metadata dataset (`pavbhaji.json`) and associated labeled image folders.

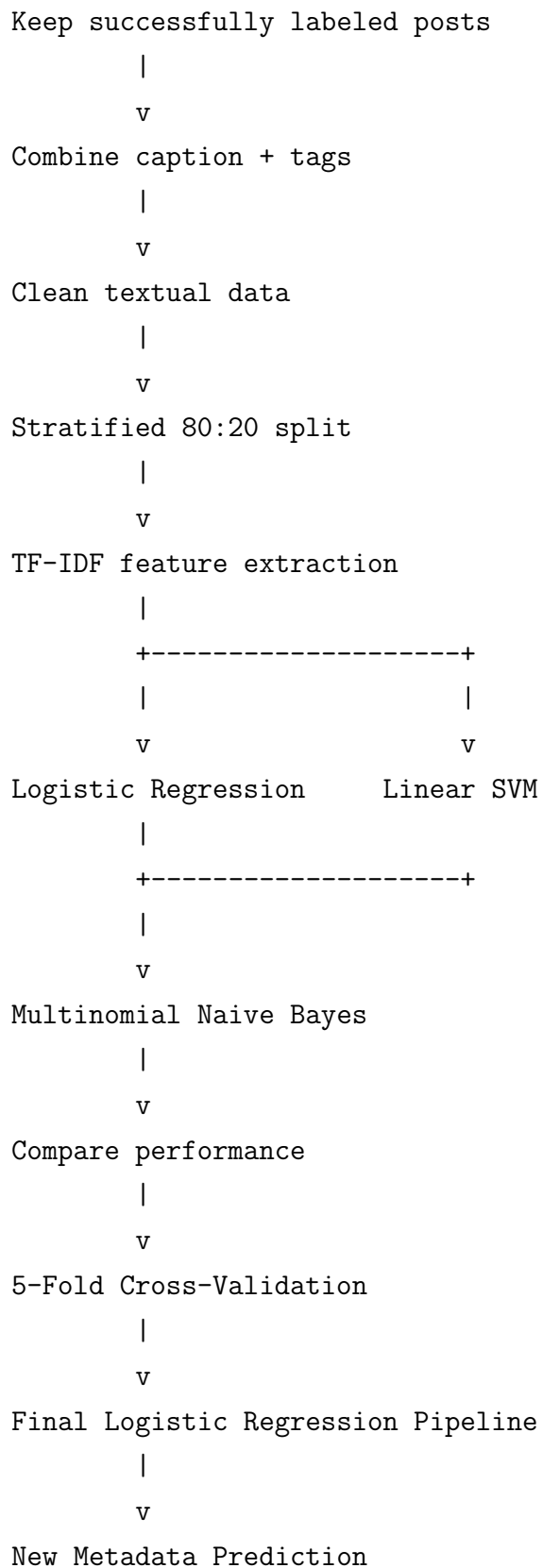
A. Appendix A: Main Implementation Workflow

The following pseudocode summarizes the complete implementation:

```

Load JSON dataset
    |
    v
Extract captions and tags
    |
    v
Match metadata with labeled images
    |
    v

```



B. Appendix B: Final Model Configuration

Table 13: Final model configuration

Component	Configuration
Input	Caption + Tags
Preprocessing	Text normalization
Feature extraction	TF-IDF
N-grams	Unigrams + Bigrams
Maximum TF-IDF features	2,000
Minimum document frequency	2
Classifier	Logistic Regression
Class weighting	Balanced
Maximum iterations	1,000
Random state	42

C. Appendix C: Performance Summary

Table 14: Final performance summary

Evaluation	Result
Test Accuracy	64.84%
Test Precision	54.72%
Test Recall	78.38%
Test F1-Score	64.44%
5-Fold CV Accuracy	68.13% $\pm$ 4.44%
5-Fold CV Precision	58.15% $\pm$ 4.33%
5-Fold CV Recall	76.46% $\pm$ 6.50%
5-Fold CV F1-Score	65.98% $\pm$ 4.75%